



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSESSMENT TOOL 3

DEBUGGING & OPTIMISATION TASK

CSA0609-DESIGN AND ANALYSIS OF ALGORITHMS

Submitted by

VENNAMPALLI BOOPATHY KHANISHK (192524311)

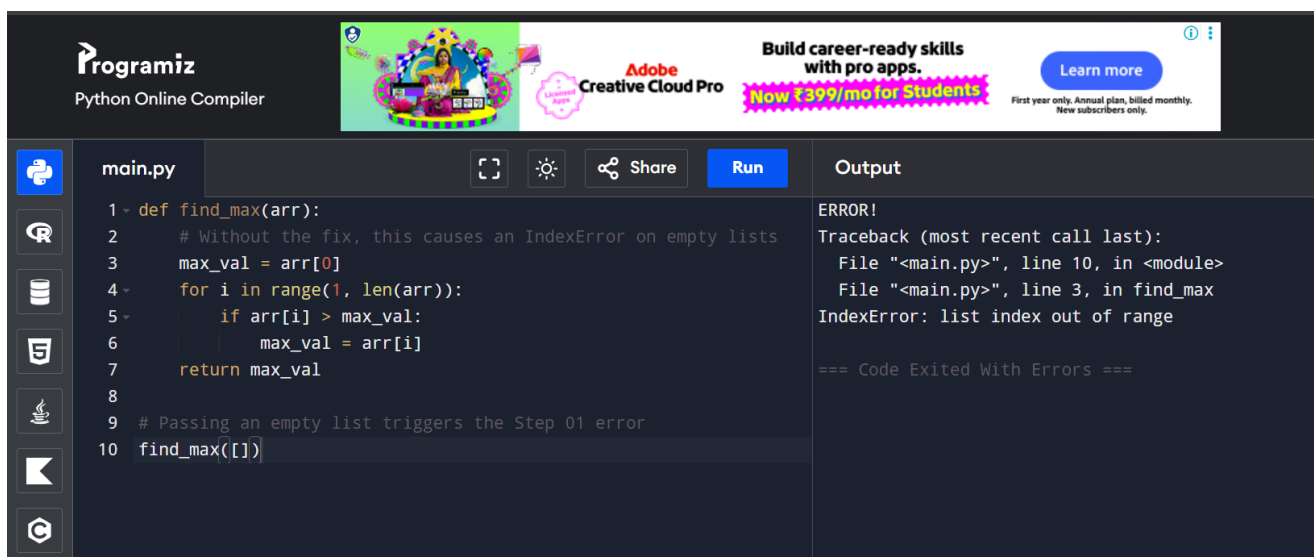
Debugging Maximum Element Search Code (Empty List and Initialization Risk)

The following code is intended to find the maximum element, but it fails for empty arrays and uses an unsafe initialization pattern that may lead to errors. Carefully analyze the assumptions made by the code and explain why robustness is affected. Apply systematic debugging to handle edge cases correctly while preserving proper maximum search behavior. Optimize the implementation for clarity and defensive programming. Analyze the time complexity of the corrected solution and rewrite the final code with suitable documentation.

```
def find_max(arr):
    max_val = arr[0] # ISSUE
    for i in range(1, len(arr)):
        if arr[i] > max_val:
            max_val = arr[i]
    return max_val
```

STEP BY STEP ERROR ANALYSIS :

STEP 01 : Direct indexing of `arr[0]` without checking if `arr` is non-empty.



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the following Python code:

```
1 def find_max(arr):
2     # Without the fix, this causes an IndexError on empty lists
3     max_val = arr[0]
4     for i in range(1, len(arr)):
5         if arr[i] > max_val:
6             max_val = arr[i]
7     return max_val
8
9 # Passing an empty list triggers the Step 01 error
10 find_max([])
```

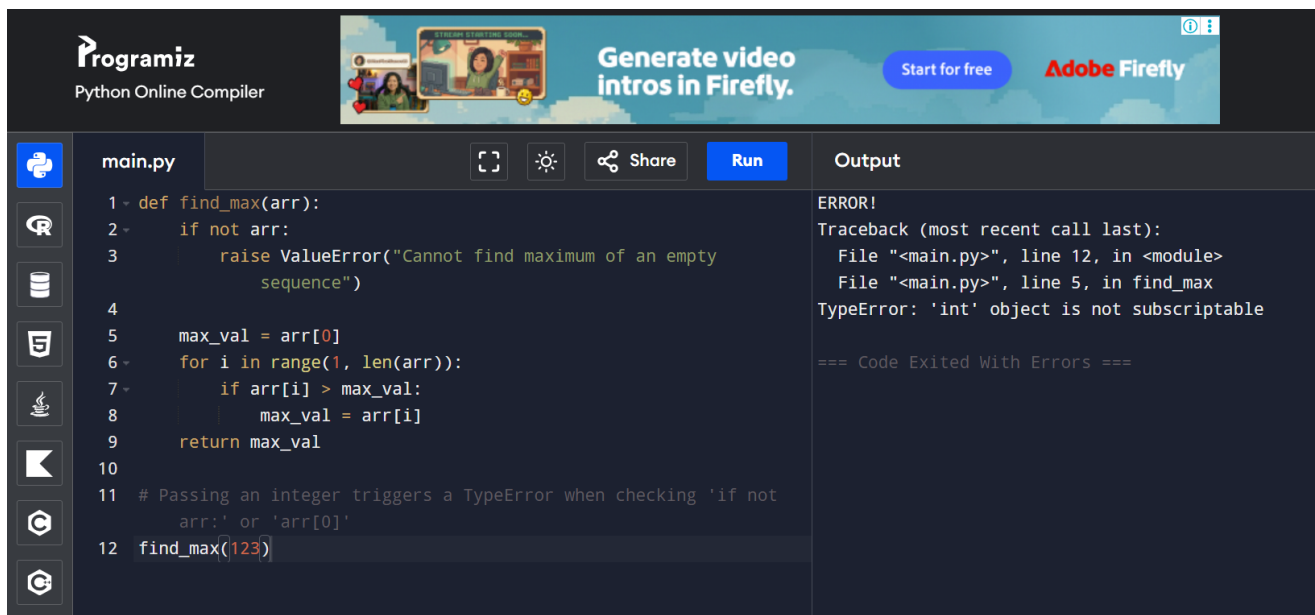
The 'Run' button is highlighted in blue. The output panel on the right displays the following error message:

```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 10, in <module>
    find_max([])
  File "<main.py>", line 3, in find_max
    max_val = arr[0]
IndexError: list index out of range

=== Code Exited With Errors ===
```

At the top of the interface, there are advertisements for Programiz, Adobe Creative Cloud Pro, and a 'Learn more' button.

STEP 02 : Strict assumption that the input `arr` is always a valid indexable sequence.



The screenshot shows the Programiz Python Online Compiler interface. The code editor contains a Python function `find_max` that checks for an empty list and then iterates through the list to find the maximum value. The function is called with the integer `123`. The output pane shows a `TypeError: 'int' object is not subscriptable` error, which occurs because the function attempts to access `arr[0]` on a non-indexable integer.

```
1- def find_max(arr):
2-     if not arr:
3-         raise ValueError("Cannot find maximum of an empty
           sequence")
4-
5-     max_val = arr[0]
6-     for i in range(1, len(arr)):
7-         if arr[i] > max_val:
8-             max_val = arr[i]
9-     return max_val
10
11 # Passing an integer triggers a TypeError when checking 'if not
   arr:' or 'arr[0]'
12 find_max(123)
```

Output:

```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 12, in <module>
    File "<main.py>", line 5, in find_max
TypeError: 'int' object is not subscriptable

=== Code Exited With Errors ===
```

EFFECTS OF ERROR :

1. If an empty list `[]` is passed into `find_max([])`, Python raises an `IndexError: list index out of range`, causing the program to crash immediately.
2. If `None` or a non-sequence data type is passed into the function, it causes an unhandled `TypeError` prior to or during length evaluation.

CORRECT STEPS :

1. Add defensive checks to handle empty input before accessing indices, such as returning `None`, raising a descriptive `ValueError`, or using a default initial value.
2. Perform type validation or ensure structural input checks so the code handles invalid input cleanly.

ALGORITHM

1. **Input:** Accept a collection `arr`.
2. **Empty / Validity Check:** Check if `arr` is `None` or empty (`len(arr) == 0`).
 - If empty or invalid, raise a `ValueError("Cannot find maximum element of an empty sequence")` or return `None` (depending on application requirements; raising `ValueError` mirrors standard Python `max()` behavior).
3. **Initialization:** Set `max_val` to the first element `arr[0]`.
4. **Iteration:** Loop through each element `item` in `arr[1:]` (or iterate directly across elements).
5. **Comparison:** If `item` is strictly greater than `max_val`, update `max_val = item`.
6. **Return Result:** Return `max_val`.

MANUAL PROCESS

Scenario A: Normal Execution (`arr = [3, 7, 2, 9, 5]`)

1. Initial check: `arr` is not empty. Proceed.
2. Initialize: `max_val = arr[0] → max_val = 3`.
3. Loop $i = 1$: `arr[1] = 7`. Compare: $7 > 3$ is True. Update `max_val = 7`.
4. Loop $i = 2$: `arr[2] = 2`. Compare: $2 > 7$ is False. Keep `max_val = 7`.
5. Loop $i = 3$: `arr[3] = 9`. Compare: $9 > 7$ is True. Update `max_val = 9`.
6. Loop $i = 4$: `arr[4] = 5`. Compare: $5 > 9$ is False. Keep `max_val = 9`.
7. End of Loop: Return 9.

Scenario B: Edge Case (`arr = []`)

1. **Initial check:** `arr` is empty (`len([]) == 0`).
2. **Defensive trigger:** Instead of attempting to evaluate `arr[0]` (which would crash), the defensive check instantly raises `ValueError("Cannot find maximum of an empty sequence")` (or returns `None`).
3. **Outcome:** The program handles the boundary condition gracefully without unhandled crashes.

COMPLEXITIES

- **Time Complexity: $O(n)$**
 - **Explanation:** The function visits every element of the array of size n exactly once during iteration to find the maximum value.
- **Space Complexity: $O(1)$**
 - **Explanation:** The memory footprint remains constant because only a single auxiliary variable (`max_val`) is used, regardless of the size of `arr`.

DEBUGGING SUMMARY :

- **Primary Flaw:** The original function relies on direct index access (`arr[0]`) without verifying whether the input sequence contains any elements.
- **Failure Scenario:** When an empty list (`[]`) is passed, evaluating `arr[0]` raises an uncaught `IndexError`, making the code fragile and prone to sudden runtime crashes.
- **Risks of Arbitrary Initializers:** Initializing `max_val` with arbitrary external values (like `0` or `float('-inf')`) introduces subtle logical bugs, particularly when dealing with lists containing negative numbers or custom objects.
- **Defensive Programming Solution:** The implementation must validate that the input array `arr` is truthy and non-empty before attempting to access any indices.
- **Explicit Fault Signaling:** Raising an explicit `ValueError` on empty inputs—following the standard behavior of Python's built-in `max()` function—or returning `None` prevents unexpected crashes.
- **Overall Outcome:** Enforcing proper input validation guarantees robust, error-free max-search execution across all boundary edge cases.

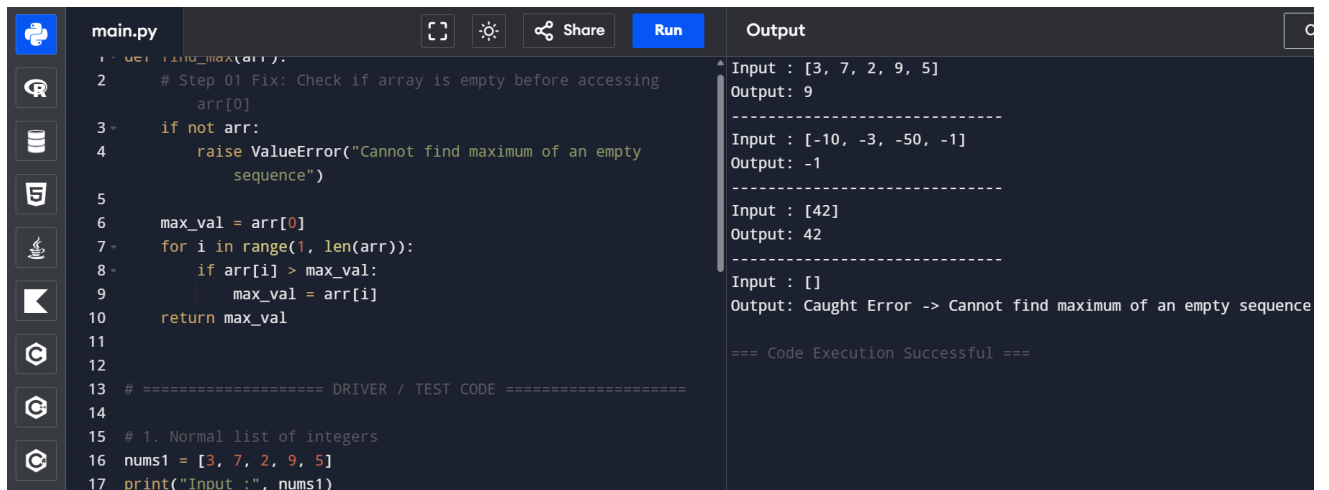
PSEUDO CODE :

```
FUNCTION find_max(arr):  
    // Check if the sequence is empty or invalid  
    IF arr IS EMPTY OR arr IS NULL THEN  
        RAISE ValueError("Cannot find maximum of an empty sequence")  
    END IF  
  
    // Initialize the max tracker with the first element  
    SET max_val TO arr[0]  
  
    // Iterate through the remaining elements  
    FOR EACH element FROM index 1 TO length(arr) - 1 DO  
        IF arr[index] > max_val THEN  
            SET max_val TO arr[index]  
        END IF  
    END FOR  
  
    RETURN max_val  
END FUNCTION
```

CORRECTED & OPTIMIZED IMPLEMENTATION

CODE :

```
def find_max(arr):  
    # Step 01 Fix: Check if array is empty before accessing arr[0]  
    if not arr:  
        raise ValueError("Cannot find maximum of an empty sequence")  
  
    max_val = arr[0]  
    for i in range(1, len(arr)):  
        if arr[i] > max_val:  
            max_val = arr[i]  
    return max_val
```



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'find_max' that checks for an empty array and then finds the maximum value. Below the function, there is a driver/test code section that tests the function with various inputs: a normal list of integers, an empty list, and a list with negative numbers. The output of the code execution is shown on the right side of the editor.

```
main.py  
1 def find_max(arr):  
2     # Step 01 Fix: Check if array is empty before accessing  
   arr[0]  
3     if not arr:  
4         raise ValueError("Cannot find maximum of an empty  
   sequence")  
5  
6     max_val = arr[0]  
7     for i in range(1, len(arr)):  
8         if arr[i] > max_val:  
9             max_val = arr[i]  
10    return max_val  
11  
12  
13 # ===== DRIVER / TEST CODE =====  
14  
15 # 1. Normal list of integers  
16 nums1 = [3, 7, 2, 9, 5]  
17 print("Input :", nums1)
```

Output

```
Input : [3, 7, 2, 9, 5]  
Output: 9  
-----  
Input : [-10, -3, -50, -1]  
Output: -1  
-----  
Input : [42]  
Output: 42  
-----  
Input : []  
Output: Caught Error -> Cannot find maximum of an empty sequence  
  
=== Code Execution Successful ===
```

DRIVER / TEST CODE

1. Normal list of integers

```
nums1 = [3, 7, 2, 9, 5]
print("Input :", nums1)
print("Output:", find_max(nums1))
print("-" * 30)
```

2. List with negative numbers

```
nums2 = [-10, -3, -50, -1]
print("Input :", nums2)
print("Output:", find_max(nums2))
print("-" * 30)
```

3. Single-element list

```
nums3 = [42]
print("Input :", nums3)
print("Output:", find_max(nums3))
print("-" * 30)
```

4. Testing the empty list edge case (raises ValueError)

```
try:
    print("Input : []")
    print("Output:", find_max([]))
except ValueError as e:
    print("Output: Caught Error ->", e)
```